ecomness

DEVELOPER DOCUMENTATION

Ecomness Public API

Read the catalogue, place orders and track them from your own systems, with an API key from your Ecomness account.

BASE URL	FOR	VERSION
https://ecomness.com/api/v1	**Sellers and suppliers**	**v1.0 · October 2026**

Content

00 Overview
01 Getting your API key
02 Authentication and conventions
03 Rate limits
04 Products
05 Placing an order
06 Tracking orders
07 Reference data and statuses
08 Errors
09 Postman collection

QUICK REFERENCE

Base URL	`https://ecomness.com/api/v1`
Your key	Integrations › Ecomness API › Manage API keys in your Ecomness dashboard, one key per account
Authentication	`x-api-key: ecm_…` or `Authorization: Bearer ecm_…`
Format	JSON in and out; success in `data`, failure in `error`
Rate limits	120 requests per minute per key, 300 per minute per IP; `429` with `Retry-After` above that
Endpoints	`GET /products`, `GET /products/:code`, `POST /orders`, `GET /orders?codes=`, `GET /reference`, `GET /statuses`

00 · OVERVIEW

What the API does

Six endpoints, one key per account. The API returns what your dashboard shows you.

METHOD	PATH	PURPOSE	WHO
GET	/products	List the catalogue you can see	Seller, supplier
GET	/products/:code	One product by its code	Seller, supplier
POST	/orders	Place an order	Seller
GET	/orders?codes=…	Track up to 50 orders or parcels at once	Seller, supplier
GET	/reference	Governorates, order sources, categories, colours	Seller, supplier
GET	/statuses	Every order status and delivery failure reason	Seller, supplier

The typical integration

Create a key → GET /products → POST /orders → GET /orders?codes=

Read the catalogue to get product and variant codes and the lowest price you may sell at, send orders with those codes, then track the order codes to follow each parcel from packing to delivery.

01 · GETTING YOUR API KEY

Getting your API key

Keys are created and deleted from the Integrations page of your Ecomness dashboard.

1. Log in to Ecomness with your seller or supplier account.
2. Open **Integrations** in the sidebar and find the **Ecomness API** card.
3. Click **Manage API keys**.
4. Give the key a name you will recognise later, for example *Shopify store*, pick how long it lasts (30, 90, 180 or 365 days) and click **Create key**.
5. The full key appears once. Copy it and store it in your system's secret settings.

THE KEY IS SHOWN ONLY ONCE

It cannot be shown again or recovered. If you lose it, delete it and create a new one.

Rules for keys

RULE	DETAIL
Who can create one	Accounts with API access enabled by Ecomness. Until it is enabled, the buttons on the Ecomness API card are unavailable.
How many	One key per account. To replace it, delete the current key, then create a new one.
Format	Starts with `ecm_` followed by 64 characters. The dashboard shows only the first characters, for example `ecm_VaKzKc…` .
Expiry	Every key expires, after 30, 90, 180 or 365 days. An expired key stops working at once and frees your slot for a new one.
Revocation	Deleting the key from the Ecomness API card stops it immediately. Ecomness can also disable a key.

02 · AUTHENTICATION AND CONVENTIONS

Authentication and conventions

Send the key on every request. Everything else is plain JSON over HTTPS.

Sending the key

Use either header. Both are accepted on every endpoint.

```
x-api-key: ecm_VaKzKcwGYUIl…

Authorization: Bearer ecm_VaKzKcwGYUIl…
```

Request and response format

- Base URL: `https://ecomness.com/api/v1` . HTTPS only.
- Request bodies are JSON with `Content-Type: application/json` , at most 64 KB.
- A successful response wraps its content in `data` . A failed one wraps it in `error` , always with a stable `code` your code can switch on and a `message` written for people.
- Codes (`ENP-` products, `ENPV-` variants, `ENO-` orders, `ENSO-` parcels) are case-insensitive.
- Prices and totals are in Tunisian dinars (TND). Dates are ISO 8601, returned in UTC.
- Responses are never cached (`Cache-Control: no-store`).

SUCCESS

```
{ "data": { … } }
```

FAILURE

```
{
  "error": {
    "code": "PRICE_TOO_LOW",
    "message": "unitPrice must be at least 55.00.",
    "details": [ … ]
  }
}
```

`details` is present only when there is more to say, for example one entry per refused order line.

03 · RATE LIMITS

Rate limits

Two limits apply to every request.

LIMIT	DEFAULT	COUNTED PER
Per key	120 requests per minute	Your API key
Per address	300 requests per minute	Your IP address

Each limit counts your requests over one minute, then starts again.

Over a limit, the API answers `429` with a `Retry-After` header giving the seconds to wait:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 23

{ "error": { "code": "RATE_LIMITED", "message": "Too many requests. Slow down and retry later." } }
```

Staying well inside the limits

- Track in batches: one `GET /orders?codes=` call takes up to 50 codes.
- Cache `/reference` and `/statuses` on your side; they rarely change.
- On `429`, wait the number of seconds in `Retry-After` before trying again.

Need a higher limit for a large integration? Contact Ecomness support: limits can be raised for a single key.

04 · PRODUCTS

Products

The catalogue as your dashboard shows it, with the codes you need to place orders.

`GET` `/products` — SELLER · SUPPLIER

QUERY PARAMETER	DESCRIPTION
`page`	Page number, from 1. Default 1.
`pageSize`	Products per page, 1 to 100. Default 20.
`search`	Part of a product name or code.
`category`	A category from `GET /reference`, for example `KITCHEN`.
`inStock`	`true` for products with stock left, `false` for stock-outs.
`approvalStatus`	Supplier accounts: `approved` or `pending`.

RESPONSE


```
{
  "data": {
    "page": 1, "pageSize": 20, "total": 4, "totalPages": 1,
    "products": [
      {
        "code": "ENP-VARIANT1",
        "name": "Lampe de bureau",
        "description": "…",
        "category": "DECORATION",
        "delivery": "24-48h",
        "reservationOnly": false,
        "stock": null,
        "media": [ { "url": "https://utfs.io/f/…", "type": "image" } ],
        "price": 20,
        "minimumUnitPrice": 22,
        "variants": [
          { "code": "ENPV-RED00001", "color": "RED", "size": "M",
            "stock": 1, "price": 20, "minimumUnitPrice": 30 },
          { "code": "ENPV-BLUE0001", "color": "BLUE", "size": "L",
            "stock": 0, "price": 22, "minimumUnitPrice": 24.2 }
        ]
      }
    ]
  }
}
```

FIELD	MEANING
`price`	Your price per unit.
`minimumUnitPrice`	The lowest `unitPrice` you may sell one unit at when you place an order. Lower prices are refused.
`stock`	Units available. `null` when the product has variants: each variant carries its own stock and price.
`reservationOnly`	`true` when the product can only be ordered from stock you reserved in the dashboard.
`variants`	Colour and size options. If a product has variants, every order line for it must name one.

Supplier accounts get the same shape without `minimumUnitPrice`, plus `approvalStatus` (`approved` once the product is published, `pending` while it is under review), `createdAt` and `updatedAt`.

`GET` `/products/:code` — SELLER · SUPPLIER

One product, in exactly the shape the list returns for you. A product you cannot see answers `404 PRODUCT_NOT_FOUND`.

```
GET https://ecomness.com/api/v1/products/ENP-VARIANT1
```

05 · PLACING AN ORDER

Placing an order

Available to seller accounts. Every line is checked before anything is created.

`POST` `/orders` — SELLER

FIELD	REQUIRED	RULES
`customer.firstName`, `lastName`	Yes	At least 2 characters each.
`customer.phone`	Yes	Exactly 8 digits.
`customer.email`	No	A valid email address.

FIELD	REQUIRED	RULES
`customer.state`	Yes	A governorate exactly as listed by `GET /reference`, for example `Ben Arous`.
`customer.city`, `address`	Yes	At least 3 characters each.
`items`	Yes	1 to 50 lines.
`items[].productCode`	Yes	A product from your catalogue.
`items[].variantCode`	When the product has variants	Must be one of the product's variants. Sent for a product without variants, it is ignored.
`items[].quantity`	Yes	Whole number, 1 to 1000.
`items[].unitPrice`	Yes	What the customer pays for one unit, in TND. At least the product's or variant's `minimumUnitPrice`.
`externalId`	No	Your own order reference, up to 100 characters. See *Retries* below.
`comment`	No	Note for delivery, up to 500 characters.
`openable`, `fragile`	No	Booleans, default `false`.
`source`	No	Where the customer came from, a value from `GET /reference`. Default `NA`.
`deliveryDate`	No	ISO 8601 date and time with a time zone, in the future.
`manageConfirmation`	No	`true` hands the order to the Ecomness confirmation service. Only if the service is enabled on your account. Default `false`.

REQUEST

```
{
  "externalId": "shop-3001",
  "customer": {
    "firstName": "Amine",
    "lastName": "Ben Salah",
    "phone": "22123456",
    "email": "amine@example.com",
```

```
    "state": "Tunis",
    "city": "La Marsa",
    "address": "12 rue de la Liberté"
  },
  "items": [
    { "productCode": "ENP-VARIANT1", "variantCode": "ENPV-RED00001",
      "quantity": 1, "unitPrice": 35 },
    { "productCode": "ENP-SIMPLE01", "quantity": 2, "unitPrice": 49 }
  ],
  "comment": "Appeler avant livraison",
  "openable": true,
  "fragile": false,
  "source": "INSTAGRAM",
  "deliveryDate": "2026-10-07T09:00:00+01:00",
  "manageConfirmation": false
}
```

Totals and delivery fees

You set the unit prices; Ecomness computes the rest. Products from different suppliers ship as separate parcels (sub-orders). Each parcel adds a delivery fee: **8 TND** when the order is a single parcel, **7.5 TND per parcel** when it has several. The order total is what the customer pays on delivery.

In the example, both products come from one supplier: 35 + 2 × 49 + 8 = **141 TND**.

RESPONSE, 201 CREATED

```
{
  "data": {
    "code": "ENO-6YT8GJFE",
    "createdAt": "2026-10-05T16:23:34.916Z",
    "openable": true,
    "fragile": false,
    "subOrders": [
      {
        "code": "ENSO-ZBS5K8DK",
        "status": { "code": "EC00", "label": "Emballage en cours",
                    "description": "En attente de l'emballage à compléter" },
        "trackingNumber": null,
        "total": 141,
        "history": [
          { "status": { "code": "EC00", "label": "Emballage en cours", "description": "…" },
            "at": "2026-10-05T16:23:34.921Z", "deliveryFailureReason": null }
        ],
        "items": [
          { "productCode": "ENP-VARIANT1", "variantCode": "ENPV-RED00001", "name": "Lampe de bureau",
            "color": "RED", "size": "M", "quantity": 1, "unitPrice": 35 },
          { "productCode": "ENP-SIMPLE01", "variantCode": null, "name": "Tapis de cuisine",
            "color": null, "size": null, "quantity": 2, "unitPrice": 49 }
        ]
      }
    ],
    "externalId": "shop-3001",
    "total": 141,
```

```
    "source": "INSTAGRAM",
    "comment": "Appeler avant livraison",
    "deliveryDate": "2026-10-07T08:00:00.000Z",
    "manageConfirmation": false,
    "customer": { "firstName": "Amine", "lastName": "Ben Salah", "phone": "22123456",
                  "email": "amine@example.com", "state": "Tunis", "city": "La Marsa",
                  "address": "12 rue de la Liberté" }
  }
}
```

A new parcel starts in `EC00` (packing), or `EC05` (waiting for confirmation) when `manageConfirmation` is `true`. Keep the `ENO-` code to track the order.

One bad line blocks the whole order

Every line is checked before anything is created. If any line fails, nothing is created and the answer lists every problem with the position of the line in `items` (from 0), so you can fix them all at once.

RESPONSE, 400 BAD REQUEST

```
{
  "error": {
    "code": "INVALID_ITEMS",
    "message": "4 item(s) cannot be ordered. Nothing was created; fix them and send
                the whole order again.",
    "details": [
      { "index": 1, "productCode": "ENP-VARIANT1", "variantCode": null,
        "code": "VARIANT_REQUIRED",
        "message": "\"ENP-VARIANT1\" comes in variants. Send the variantCode of the one ordered." },
      { "index": 2, "productCode": "ENP-VARIANT1", "variantCode": "ENPV-BLUE0001",
        "code": "OUT_OF_STOCK",
        "message": "Not enough stock for this quantity." },
      { "index": 3, "productCode": "ENP-NOPE0000", "variantCode": null,
        "code": "PRODUCT_NOT_FOUND",
        "message": "No product you can order has the code \"ENP-NOPE0000\"." },
      { "index": 4, "productCode": "ENP-SUPTWO01", "variantCode": null,
        "code": "PRICE_TOO_LOW",
        "message": "unitPrice must be at least 55.00." }
    ]
  }
}
```

LINE CODE	WHAT TO DO
`PRODUCT_NOT_FOUND`	The code is wrong, or the product is not published or not available to you. Check `GET /products`.
`VARIANT_REQUIRED`	The product has variants: send `variantCode`.
`VARIANT_NOT_FOUND`	The variant code does not belong to this product.

LINE CODE	WHAT TO DO
`DUPLICATE_LINE`	The same product and variant appear twice: send one line with the total quantity.
`PRICE_TOO_LOW`	Raise `unitPrice` to at least `minimumUnitPrice`.
`OUT_OF_STOCK`	Not enough stock for the quantity. Lower it or drop the line.
`RESERVATION_REQUIRED`	Reservation-only product, and your reserved stock does not cover the quantity. Reserve stock in the dashboard first.

If the stock runs out while the order is being placed, you get the same `INVALID_ITEMS` answer and nothing is created.

Retries: use externalId

If a request times out, you cannot tell whether the order was created. Send your own order reference as `externalId`: when the same reference arrives again, Ecomness refuses it and names the order already created, so a retry can never place the order twice.

RESPONSE, 409 CONFLICT

```
{
  "error": {
    "code": "DUPLICATE_EXTERNAL_ID",
    "message": "An order with externalId \"shop-3001\" already exists: ENO-6YT8GJFE.
                Track it instead of sending it again."
  }
}
```

06 · TRACKING ORDERS

Tracking orders

Follow up to 50 orders or parcels in one call.

`GET` `/orders?codes=ENO-…,ENSO-…` SELLER · SUPPLIER

`codes` is a comma-separated list of order codes (`ENO-`) and parcel codes (`ENSO-`), mixed freely, at most 50. You only ever get your own: a code that is not yours comes back in `notFound`, exactly like a code that does not exist.

REQUEST

```
GET https://ecomness.com/api/v1/orders?codes=ENO-OMC2PZJY,ENO-NOTMINE1
```

RESPONSE


```
{
  "data": {
    "orders": [
      {
        "code": "ENO-OMC2PZJY",
        "…": "same shape as the POST /orders response",
        "subOrders": [
          {
            "code": "ENSO-KRTTJOUU",
            "status": { "code": "106", "label": "Problème de livraison",
                        "description": "…" },
            "trackingNumber": "BW-4471209",
            "total": 56.5,
            "history": [
              { "status": { "code": "EC00", "label": "Emballage en cours", "description": "…" },
                "at": "2026-10-05T16:07:18.553Z", "deliveryFailureReason": null },
              { "status": { "code": "106", "label": "Problème de livraison", "description": "…" },
                "at": "2026-10-07T11:40:02.000Z",
                "deliveryFailureReason": { "code": "33", "label": "Ne répond pas" } }
            ],
            "items": [ … ]
          }
        ]
      }
    ],
    "notFound": ["ENO-NOTMINE1"]
  }
}
```


Supplier accounts see the parcels they ship in each order.

Asking by order code returns all your parcels in that order; asking by parcel code returns that parcel. `trackingNumber` is the delivery company's parcel number once it is shipped.

07 · REFERENCE DATA AND STATUSES

Reference data and statuses

The values requests accept and responses use. Cache them; they change rarely.

GET /reference — SELLER · SUPPLIER


```
{
  "data": {
    "governorates": [
      "Ariana", "Béja", "Ben Arous", "Bizerte", "Gabès", "Gafsa",
      "Jendouba", "Kairouan", "Kasserine", "Kébili", "Kef", "Mahdia",
      "Manouba", "Médenine", "Monastir", "Nabeul", "Sfax", "Sidi Bouzid",
      "Siliana", "Sousse", "Tataouine", "Tozeur", "Tunis", "Zaghouan"
    ],
    "orderSources": [
```

```
      "FB", "INSTAGRAM", "TIKTOK", "TEL", "SHOPIFY", "WOOCOMMERCE",
      "CONVERTY", "TIKTAKPRO", "YOUCAN", "LIGHTFUNNEL", "WHATSAPP", "NA"
    ],
    "productCategories": [
      "FURNITURE", "DECORATION", "CAR_ACCESSORIES", "HOME_APPLIANCES",
      "ELECTRONICS", "CLOTHING", "BEAUTY", "SPORTS", "TOYS", "JEWELRY",
      "GARDEN", "KITCHEN", "ACCESSORIES"
    ],
    "colors": [
      "RED", "BLUE", "GREEN", "YELLOW", "BLACK", "WHITE", "ORANGE",
      "PURPLE", "PINK", "LIGHT_BLUE", "NAVY_BLUE", "TURQUOISE", "BEIGE",
      "BROWN", "GRAY", "LIGHT_PINK", "GOLD", "BRONZE", "SALMON_RED",
      "BURGUNDY_RED", "WATER_GREEN"
    ]
  }
}
```

`GET` `/statuses` SELLER · SUPPLIER

Every status a parcel can be in, with the French label and description your dashboard shows, and every delivery failure reason. Statuses belong to parcels (`ENSO-`): an order can ship as several parcels, each with its own status.

CODE	LABEL	MEANING
Confirmation, only for orders sent with `manageConfirmation: true`		
`EC05`	Attente de confirmation	Waiting for the customer to be called and confirm.
`EC06` to `EC10`	Tentative de confirmation 1 to 5	Confirmation call attempts 1 to 5.
Before pickup		
`EC00`	Emballage en cours	Confirmed, being packed.
`EC01`	Annulée	Cancelled before packing.
Delivery		
`101`	En attente d'enlèvement	Packed, waiting for the delivery company.
`102`	Colis enlevé	Picked up, in transit.
`103`	Problème d'enlèvement	Problem during pickup.
`104`	Réception dépôt	Arrived at the delivery depot.

CODE	LABEL	MEANING
105	En cours de livraison	Out for delivery.
106	Problème de livraison	Delivery problem; see the failure reason.
107	Colis Livré	Delivered to the customer.
108	À retourner	Return scheduled.
109	Colis Retourné	Returned.
Settled		
EC02	Livrée payée	Delivered and paid out.
EC03	Retournée payée	Returned and settled.

Delivery failure reasons

When a delivery fails, the history entry carries a reason such as `{ "code": "33", "label": "Ne répond pas" }`.

CODE	LABEL	CODE	LABEL
33	Ne répond pas	44	Commande annulée
34	Injoignable	45	Colis en double
36	Rendez-vous Demain	46	Faute de manipulation
37	Refusé par le client	47	Rendez-vous Demain
38	Manque d'argent	48	Colis daté
39	Client n'est pas serieux	49	Client Non Disponible
40	Modalité de paiement refuser	50	Force Majeur/Colis Planifier Pour demain
41	Changement d'adresse	51	Problème Douane
42	Numéro de téléphone incorrect	52	Manque Document (Facture/BL)
43	Commande non conforme	53	Adresse erronée

Errors

Switch on `error.code`, never on the message: codes are stable, messages may be reworded.

STATUS	ERROR.CODE	MEANING
400	`VALIDATION_ERROR`	The request does not match the expected shape. `details` lists each field (`path`) and what is wrong.
400	`INVALID_JSON`	The body is not valid JSON.
400	`INVALID_ITEMS`	One or more order lines cannot be ordered; nothing was created. See section 05.
400	`CONFIRMATION_NOT_ENABLED`	`manageConfirmation` sent while the service is not enabled on your account.
400	`DELIVERY_DATE_NOT_IN_FUTURE`	`deliveryDate` is in the past.
401	`MISSING_API_KEY`	No key in `x-api-key` or `Authorization`.
401	`INVALID_API_KEY`	The key does not exist, or was deleted.
401	`API_KEY_EXPIRED`	The key reached its expiry date. Create a new one.
401	`API_KEY_DISABLED`	This key has been disabled.
403	`ACCOUNT_NOT_ALLOWED`	The account linked to this key cannot use the API right now.
403	`NOT_ALLOWED`	This key cannot use this endpoint.
404	`PRODUCT_NOT_FOUND`	No product you can see has this code.
409	`DUPLICATE_EXTERNAL_ID`	An order with this `externalId` already exists; the message names it.
413	`BODY_TOO_LARGE`	The body is over 64 KB.
429	`RATE_LIMITED`	Too many requests. Wait the seconds in `Retry-After`.
500	`INTERNAL_ERROR`	Something failed on our side. Retry later; for orders, retry with the same `externalId`.

09 · POSTMAN COLLECTION

Postman collection

Every endpoint, ready to run, in `ecomness-api.postman_collection.json`.

1. Download the collection from the **Ecomness API** card (**Manage API keys** › **Postman collection**). In Postman, choose **Import** and select the file.
2. Open the collection, go to **Variables**, and paste your key into `apiKey`. Keep `baseUrl` as `https://ecomness.com/api/v1`.
3. Run **Products › List products** first. It saves the first product's code, and its first variant's code, into the collection variables `productCode` and `variantCode`.
4. Run **Orders › Place an order**. Adjust `unitPrice` to at least the product's `minimumUnitPrice`. On success it saves the new order code into `orderCode`.
5. Run **Orders › Track orders** to follow that order, and the two **Reference** requests for governorates and statuses.

Every request uses the key from the collection's authorization, so it is set in one place.